

■ 2.2 Transformation Rules and Definitions

■ 2.2.1 Evaluating Expressions

The operation of *Mathematica* can be summarized by saying that when you type in an expression, *Mathematica* evaluates it to give you a result. The process of evaluation can involve many different kinds of operations. If you typed in the expression $6 + 7$, evaluation would consist in applying a procedure for adding integers, to give you the result 13. If you typed in $x - 3x + 1$, rules for simplification would be applied to give you the result $1 - 2x$. If you had set $x = 5$, then x would be replaced by 5, and $1 - 2x$ would be reduced to -9.

Section 2.1 discussed how all the different kinds of objects that *Mathematica* deals with are represented in one uniform way: as expressions. The operations that *Mathematica* performs to evaluate expressions can also be viewed in a uniform way. All the operations consist of applying *transformation rules* to change expressions from one form to another.

There are many different kinds of transformation rules. Section 1.4.1 mentioned some rules that *Mathematica* uses to carry out algebraic simplifications on sums of terms. Definitions of functions can also be thought of as rules. So, for example, $f[x_] := x^2$ can be thought of as defining a rule for transforming all expressions of the form $f[anything]$. Later, we will discuss how you can add many different kinds of transformation rules to *Mathematica*.

When *Mathematica* evaluates an expression, it goes on applying transformation rules until it can find no more rules to apply. The *result* of the evaluation is the expression produced at this point.

The result of a particular evaluation depends on the transformation rules that *Mathematica* “knows” when it does the evaluation. If you add new rules, *Mathematica* may be able to take the evaluation of a particular expression further.

When *Mathematica* evaluates an expression like $2 + x$, its first step is to try and find any value that you may have defined for x . Assuming there is no value for x , *Mathematica* then tries to find transformation rules for the expression $2 + x$. None of the rules for sums of terms discussed in Section 1.4.1 apply, so the expression $2 + x$ will be the final result. If you subsequently assigned a value to x , this expression could be evaluated further. Without a value specified for x , *Mathematica* will, however, give the “symbolic” result $2 + x$, which involves the “undetermined parameter” x . As we have discussed at some length, one of the most crucial features of *Mathematica* is its ability to produce this kind of symbolic result.

If you type in the expression $f[3]$, *Mathematica* will again try to find rules

to evaluate it. If you had given a definition for $\mathbf{f}[\mathbf{x}_-]$, then *Mathematica* would use this definition. But if it can find no such definition, *Mathematica* leaves the expression $\mathbf{f}[3]$ unchanged, just as it leaves the symbol $\mathbf{x}$ unchanged in $2 + \mathbf{x}$.

Mathematica evaluates $3 + 5$ to give 8, but can then find no rules for $\mathbf{f}[8]$, so leaves it unchanged.

```
In[1]:= f[3 + 5]
Out[1]= f[8]
```

Mathematica finds no rules for $\mathbf{f}[\text{anything}]$, but still applies algebraic simplification rules to collect together the $\mathbf{f}[4]$ terms.

```
In[2]:= f[4] + 3 f[4] + 2 f[5]
Out[2]= 4 f[4] + 2 f[5]
```

Once again $\mathbf{f}$ is treated as an “unknown function”.

```
In[3]:= t = f[2] + f[x]
Out[3]= f[2] + f[x]
```

This specifies a transformation rule for $\mathbf{f}[\text{anything}]$.

```
In[4]:= f[x_] = x^2
Out[4]= x^2
```

Now the expression $\mathbf{t}$ defined above can be evaluated further, using the transformation rule you have defined for $\mathbf{f}[\mathbf{x}_-]$.

```
In[5]:= t
Out[5]= 4 + x^2
```